QA STUDY LIBRARY

Database & SQL

for QA — From Zero to Viva-Ready

Everything in simple words — tables, SELECT, WHERE, JOINs, GROUP BY, subqueries, INSERT/UPDATE/DELETE, and real database-testing scenarios.

Quick-scan guide for SQA interviews and vivas

Includes for your interview:

9 chapters (basic → advanced) + a query cheat-sheet

+ 35 viva questions with answers, all QA-focused

What's inside

How to use this guide

Read Parts 1–4 to build the muscle, Part 5 (JOINs) is the one interviewers love, Part 8 is where QA meets SQL, and Part 9 is your night-before-the-viva revision.

Every query here is standard SQL that runs on MySQL / PostgreSQL / SQL Server with tiny tweaks.

PART 1 — Database basics — the ground floor

What a database really is, and why a tester must speak SQL

1.1 What is a database? (the filing-cabinet story)

Picture a giant, very organised filing cabinet. Each **drawer** holds one kind of thing — one drawer for customers, one for orders, one for products. Inside a drawer, every customer gets one **index card**, and every card has the same labelled boxes: name, email, phone. A **database** is that cabinet; a **table** is a drawer; a **row** is one card; a **column** is one labelled box on every card.

SQL (Structured Query Language) is simply how you *talk* to the cabinet: “bring me every customer from Dhaka,” “count the orders from yesterday,” “change this email.” You ask in one sentence and the database does the searching.

1.2 The words you'll hear every day

Term	Plain meaning	Cabinet picture
Table	A collection of rows about one thing	A drawer (Customers)
Row / Record	One single item	One index card
Column / Field	One property that every row has	A labelled box (email)
Primary Key (PK)	The unique ID of a row — never repeats, never empty	The card's ID number
Foreign Key (FK)	A column that points to a row in another table	“belongs to customer #12”
Schema	The blueprint: which tables/columns exist	The cabinet's floor plan
Query	A question you send in SQL	“fetch all Dhaka cards”

1.3 Primary key vs foreign key (a viva favourite)

A **primary key** uniquely identifies each row in its own table (e.g. `customer_id`). A **foreign key** is a column in one table that references the primary key of another, which is how tables link. In an `orders` table, `customer_id` is a foreign key pointing back at the `customers` table — that's how you know *whose* order it is.

> **Viva favourite — PK vs FK**
>
> “A primary key is unique and identifies a row inside one table. A foreign key lives in another table and points to that primary key, creating the relationship between them. Foreign keys are what let me JOIN two tables together.”

1.4 Relational vs non-relational (SQL vs NoSQL)

	Relational (SQL)	Non-relational (NoSQL)
Shape	Tables with fixed columns	Documents / key-value / graphs
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Redis, Cassandra

	Relational (SQL)	Non-relational (NoSQL)
Best for	Structured, related data (banking, orders)	Flexible or huge-scale data
Language	SQL	Varies (Mongo query, etc.)

For QA interviews, “relational” is the default assumption. If asked the difference, the one-liner is: *relational stores data in linked tables with a strict schema; NoSQL stores flexible documents without a fixed schema.*

1.5 Why does a tester need SQL?

This is almost always asked. Keep these reasons ready:

- **Back-end verification:** the UI says “profile updated” — but did the database actually change? One SELECT proves it.
- **Test data setup & teardown:** insert the exact rows a test needs, then clean them up.
- **Bug investigation:** reproduce and confirm data-level defects (duplicate rows, wrong totals).
- **Data integrity:** check no orphan records, no broken foreign keys, no NULLs where they're banned.
- **ETL / migration testing:** compare source vs destination counts and values after data moves.

> **One-line answer**
>
> “As a tester I use SQL to verify at the database level what the application claims at the UI level — to set up test data, and to confirm data integrity after every create, update, or delete.”

PART 2 — SELECT — reading data

The one command you'll use 90% of the time

2.1 The shape of a SELECT

SELECT reads data without changing anything — the tester's safest and most-used command. The skeleton, in the order the database reads it, is:

```
SELECT   column1, column2      -- WHICH columns you want
FROM     table_name           -- WHICH table
WHERE    condition            -- WHICH rows (filter)
ORDER BY column1 ASC/DESC      -- sort the result
LIMIT    10;                   -- cap how many rows come back
```

Only `SELECT` and `FROM` are required; the rest are optional. `SELECT *` means “every column.”

2.2 Your first queries

```
-- Everything from the customers table
SELECT * FROM customers;

-- Only two columns
SELECT first_name, email FROM customers;

-- One customer by id
SELECT * FROM customers WHERE customer_id = 12;
```

2.3 ORDER BY, LIMIT, DISTINCT, AS

Clause	What it does	Example
ORDER BY	Sorts results (ASC = A→Z, DESC = Z→A)	ORDER BY created_at DESC
LIMIT	Returns only the first N rows	LIMIT 5
DISTINCT	Removes duplicate values	SELECT DISTINCT city
AS	Renames a column/table (alias) in the output	SELECT price AS cost

```
-- 5 newest orders, most recent first
SELECT order_id, total, created_at
FROM orders
ORDER BY created_at DESC
LIMIT 5;

-- Unique list of cities customers live in
SELECT DISTINCT city FROM customers ORDER BY city;

-- Alias makes output readable
SELECT first_name AS name, total AS "Order Total" FROM orders;
```

Tester tip

In SQL Server the row cap is `SELECT TOP 5 ...` instead of `LIMIT 5`. In Oracle (older) it's `ROWNUM`. MySQL and PostgreSQL both use `LIMIT`. Mentioning this shows real awareness.

PART 3 — Filtering like a pro

WHERE is where testing lives — narrowing to the exact rows

3.1 Comparison & logical operators

Operator	Meaning	Example
= / <>	equal / not-equal	status <> 'cancelled'
> < >= <=	greater / less than	total >= 1000
AND	both conditions true	city='Dhaka' AND active=1
OR	either condition true	city='Dhaka' OR city='Khulna'
NOT	reverses a condition	NOT (status='paid')

The bracket trap (interviewers test this)

`AND` is evaluated before `OR`. So `WHERE a=1 OR b=2 AND c=3` is read as `a=1 OR (b=2 AND c=3)`. Always add brackets to mean what you intend: `WHERE (a=1 OR b=2) AND c=3`.

3.2 IN, BETWEEN, LIKE

```
-- IN: matches any value in a list (cleaner than many ORs)
SELECT * FROM customers WHERE city IN ('Dhaka','Khulna','Sylhet');

-- BETWEEN: a range, inclusive of both ends
SELECT * FROM orders WHERE total BETWEEN 500 AND 1500;

-- LIKE: pattern match. % = any characters, _ = exactly one character
SELECT * FROM customers WHERE email LIKE '%@gmail.com';   -- ends with
SELECT * FROM customers WHERE first_name LIKE 'A%';       -- starts with A
SELECT * FROM products  WHERE code LIKE 'SKU_2_';         -- SKU + any + 2 + any
```

3.3 The NULL trap — the classic gotcha

NULL means “no value / unknown.” It is *not* zero and *not* an empty string. The catch: you cannot test it with =. `WHERE phone = NULL` returns nothing, always. You must use `IS NULL` / `IS NOT NULL`.

```
-- WRONG: always returns 0 rows
SELECT * FROM customers WHERE phone = NULL;

-- RIGHT
SELECT * FROM customers WHERE phone IS NULL;      -- missing phones
SELECT * FROM customers WHERE phone IS NOT NULL;  -- have a phone
```

Viva favourite — why NULL matters to QA

“NULL is unknown, so any comparison with it is unknown, not true — that's why = `NULL` fails and you must use `IS NULL`. As a tester I check NULLs because a required field that's silently NULL is a real data-integrity bug.”

PART 4 — Functions & grouping

Turning rows into numbers — counts, totals, and summaries

4.1 Aggregate functions

Aggregates squash many rows into one answer:

Function	Answers the question	Example
COUNT()	How many rows?	COUNT(*) / COUNT(email)
SUM()	What's the total?	SUM(total)
AVG()	What's the average?	AVG(total)
MIN() / MAX()	Smallest / largest value	MAX(created_at)

```
SELECT COUNT(*)        AS total_customers FROM customers;
SELECT COUNT(phone)   AS have_phone      FROM customers;  -- NULLs are skipped
SELECT SUM(total)     AS revenue         FROM orders;
SELECT AVG(total)     AS avg_order       FROM orders;
SELECT MAX(created_at) AS newest_order   FROM orders;
```

Gotcha

`COUNT(*)` counts *all* rows. `COUNT(column)` counts only rows where that column is **not NULL**. The difference between the two is a fast way to count missing values.

4.2 GROUP BY — aggregate per category

GROUP BY splits rows into buckets and runs the aggregate on each bucket. “How many customers *per city*?” “Revenue *per month*?”

```
-- Customers per city
SELECT city, COUNT(*) AS customer_count
FROM customers
GROUP BY city
ORDER BY customer_count DESC;

-- Revenue per customer
SELECT customer_id, SUM(total) AS spent
FROM orders
GROUP BY customer_id;
```

4.3 HAVING — filtering groups

WHERE filters individual rows *before* grouping. **HAVING** filters the *groups* after aggregation. Rule of thumb: if the condition uses an aggregate like `COUNT()` or `SUM()`, it belongs in `HAVING`.

```
-- Only cities that have more than 10 customers
SELECT city, COUNT(*) AS n
FROM customers
WHERE active = 1          -- row filter (before grouping)
GROUP BY city
HAVING COUNT(*) > 10      -- group filter (after grouping)
ORDER BY n DESC;
```

Viva favourite — WHERE vs HAVING

“WHERE filters rows before they're grouped and can't use aggregate functions. HAVING filters the grouped results and is the only place you can put a condition on COUNT, SUM, AVG, etc.”

4.4 Handy text & date functions

Need	Function (MySQL flavour)	Example result
Join strings	CONCAT(a,' ',b)	'Abrar Ragib'
Upper / lower	UPPER(x) / LOWER(x)	'DHAKA'
Trim spaces	TRIM(x)	'hello'
Length	LENGTH(x)	5
Today's date	CURRENT_DATE / NOW()	2026-07-07
Part of a date	YEAR(d), MONTH(d), DAY(d)	2026

PART 5 — JOINs — combining tables

The topic interviewers love most — master this one

5.1 Why joins exist

Real data is split across tables to avoid repetition. `customers` holds people; `orders` holds purchases with a `customer_id` pointing back. To answer "what did *Abrar* buy?" you must stitch the two together — that stitching is a **JOIN**, matching a foreign key to a primary key.

```
-- The core pattern: match orders.customer_id to customers.customer_id
SELECT c.first_name, o.order_id, o.total
FROM customers c
JOIN orders o  ON o.customer_id = c.customer_id;
```

Here c and o are table aliases, and `ON` states how the rows match.

5.2 The four joins, in one picture

Join type	Keeps...	Tester meaning
INNER JOIN	Only rows that match in BOTH tables	Customers who HAVE orders
LEFT JOIN	ALL left rows + matches (NULLs if none)	All customers, even with 0 orders
RIGHT JOIN	ALL right rows + matches	All orders, even if customer missing
FULL OUTER JOIN	Everything from both, matched where possible	Find gaps on either side

```
-- INNER: only customers who placed at least one order
SELECT c.first_name, o.total
FROM customers c
INNER JOIN orders o ON o.customer_id = c.customer_id;

-- LEFT: every customer; order columns are NULL if they never ordered
SELECT c.first_name, o.total
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id;
```

5.3 A LEFT JOIN is a QA superpower

LEFT JOIN + `IS NULL` finds "orphans" and gaps — a test scenario on its own. Example: customers who registered but never ordered, or order rows pointing at a customer that no longer exists (a broken foreign key = a real bug).

```
-- Customers who have NEVER placed an order
SELECT c.customer_id, c.first_name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- Orphan orders: order points to a customer that doesn't exist
SELECT o.order_id
FROM orders o
LEFT JOIN customers c ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

> **Viva favourite — INNER vs LEFT**
>
> “INNER JOIN returns only rows that match in both tables. LEFT JOIN returns all rows from the left table and fills the right side with NULL when there's no match. I use LEFT JOIN + IS NULL to hunt for missing or orphaned records — a common data-integrity test.”

5.4 Self-join (bonus points)

A **self-join** joins a table to itself — useful when a row references another row in the same table, like an employee whose `manager_id` points to another employee.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.employee_id = e.manager_id;
```

PART 6 — Changing data safely

INSERT, UPDATE, DELETE — powerful, and easy to get wrong

6.1 INSERT — add rows

```
INSERT INTO customers (first_name, email, city)
VALUES ('Abrar', 'abrar@example.com', 'Dhaka');

-- Insert several rows at once
INSERT INTO customers (first_name, city) VALUES
  ('Rita', 'Khulna'),
  ('Sami', 'Sylhet');
```

6.2 UPDATE — change existing rows

```
UPDATE customers
SET city = 'Chattogram'
WHERE customer_id = 12;    -- <<< the WHERE is what keeps you safe
```

6.3 DELETE — remove rows

```
DELETE FROM customers WHERE customer_id = 12;
```

The rule that saves careers

An UPDATE or DELETE with no WHERE hits EVERY row in the table. `DELETE FROM customers;` empties the whole table. Before running either, first run the same filter as a SELECT (`SELECT * FROM customers WHERE customer_id = 12;`) to see exactly which rows you're about to change. On real/production data, wrap it in a transaction so you can roll back.

6.4 Transactions — all-or-nothing

A **transaction** groups several statements so they either *all* succeed or *all* undo. Essential when a test must leave the database exactly as it found it.

```
BEGIN;                       -- start
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;                      -- save both, or...
ROLLBACK;                    -- undo everything if something looked wrong
```

Tester habit

For test data, a clean pattern is: **setup** (INSERT the rows the test needs) → **run the test** → **teardown** (DELETE exactly those rows). This keeps every test run independent and repeatable.

PART 7 — Advanced SQL

Subqueries, sets, conditional logic, views, and indexes

7.1 Subqueries — a query inside a query

A subquery runs first, and its result feeds the outer query. Read it inside-out.

```
-- Customers who spent more than the average order value
SELECT first_name, total
FROM orders
WHERE total > (SELECT AVG(total) FROM orders);

-- Customers who placed at least one order (using IN)
SELECT first_name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);
```

7.2 EXISTS — “is there at least one?”

```
SELECT c.first_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

7.3 UNION — stack two result sets

`UNION` stacks the rows of two SELECTs (same columns) and removes duplicates; `UNION ALL` keeps duplicates and is faster.

```
SELECT email FROM customers
UNION
SELECT email FROM newsletter_subscribers;
```

7.4 CASE — if/else inside SQL

```
SELECT order_id, total,
       CASE
         WHEN total >= 1000 THEN 'High'
         WHEN total >= 500  THEN 'Medium'
         ELSE 'Low'
       END AS bucket
FROM orders;
```

7.5 Views & indexes (know the definition)

Concept	One-line definition	Why QA cares
View	A saved SELECT that acts like a virtual table	Simplifies repeated test queries
Index	A lookup structure that speeds up searches	Explains slow vs fast queries in perf tests
Stored procedure	A saved, reusable block of SQL	May itself be the thing under test

```
CREATE VIEW dhaka_customers AS
SELECT customer_id, first_name FROM customers WHERE city = 'Dhaka';

-- then query it like a table
SELECT * FROM dhaka_customers;
```

PART 8 — Database testing in practice

Where SQL knowledge becomes actual QA work

8.1 What is database testing?

Database testing checks that data is stored, changed, and retrieved **correctly and completely** — independent of what the UI shows. It covers four things: the data itself, its integrity, its rules (constraints), and the speed of getting it.

Focus	The question you're answering	Typical SQL check
CRUD validation	Did create/read/update/delete really persist?	SELECT after each UI action
Data integrity	No orphans, no broken foreign keys?	LEFT JOIN ... IS NULL
Data mapping	UI field → correct DB column & type?	Compare value vs column
Constraints	PK unique? NOT NULL respected? defaults set?	GROUP BY / IS NULL checks
Completeness	Row counts match expectations?	COUNT(*) source vs target

8.2 The CRUD verification pattern

The everyday loop of a QA using SQL: do an action in the app, then confirm it in the database.

```
-- CREATE: registered 'Abrar' in the UI, now prove the row exists
SELECT * FROM customers WHERE email = 'abrar@example.com';

-- UPDATE: changed city to Chattogram in the UI, confirm it saved
SELECT city FROM customers WHERE email = 'abrar@example.com';   -- expect 'Chattogram'

-- DELETE: deactivated the account, confirm the row is gone (or flagged)
SELECT COUNT(*) FROM customers WHERE email = 'abrar@example.com'; -- expect 0
```

Why not just trust the UI?

A UI can show “Saved!” while the write silently fails, or save to the wrong column, or trim/round the value. The database is the source of truth — a SELECT is the only real proof. This is the single most important reason a tester learns SQL.

8.3 Data-integrity checks you can name in a viva

- **Duplicate detection:** `GROUP BY email HAVING COUNT(*) > 1` finds duplicate accounts.
- **Orphan / broken FK:** `LEFT JOIN ... WHERE parent IS NULL` finds child rows with no valid parent.
- **Required-field NULLs:** `WHERE required_col IS NULL` finds rows breaking a NOT-NULL rule.
- **Count reconciliation:** compare `COUNT(*)` before vs after a migration or ETL run.
- **Referential rules:** confirm deleting a customer also handles their orders (cascade) or is blocked.

8.4 ACID — the reliability promise

Letter	Property	Plain meaning
A	Atomicity	All steps of a transaction happen, or none do
C	Consistency	Data always obeys the rules before & after
I	Isolation	Concurrent transactions don't corrupt each other
D	Durability	Once committed, it survives a crash

Viva-ready

“ACID stands for Atomicity, Consistency, Isolation, Durability — the four guarantees that keep a relational database reliable, especially during concurrent transactions like two people booking the last seat at once.”

PART 9 — 35 viva questions with answers

Rapid-fire revision — the night before the interview

Q1. What is a primary key?

A. A column (or set of columns) that uniquely identifies each row. It's unique and can never be NULL.

Q2. What is a foreign key?

A. A column that references the primary key of another table, creating the relationship that lets you JOIN them.

Q3. Difference between primary key and unique key?

A. Both enforce uniqueness, but a table has only one primary key (never NULL), while it can have many unique keys, and a unique key may allow one NULL.

Q4. What does SELECT * do?

A. Returns every column of the matching rows. Fine for exploring, but in real queries you should name only the columns you need.

Q5. Difference between WHERE and HAVING?

A. WHERE filters individual rows before grouping and can't use aggregates; HAVING filters grouped results and is where aggregate conditions like COUNT(*) > 5 go.

Q6. Difference between DELETE, TRUNCATE and DROP?

A. DELETE removes selected rows (can have WHERE, can roll back). TRUNCATE removes all rows fast, resets the table. DROP removes the entire table structure too.

Q7. What is the difference between INNER JOIN and LEFT JOIN?

A. INNER returns only rows matching in both tables; LEFT returns all rows from the left table with NULLs where the right has no match.

Q8. How do you find customers with no orders?

A. LEFT JOIN orders on the customer key, then WHERE the order key IS NULL — the unmatched left rows are customers with no orders.

Q9. What is a self-join?

A. Joining a table to itself, used when a row references another row in the same table, like an employee's manager_id.

Q10. Why can't you use = NULL?

A. NULL means unknown, so any comparison with it is unknown rather than true. You must use IS NULL or IS NOT NULL.

Q11. Difference between NULL, 0 and an empty string?

A. NULL is 'no value / unknown'; 0 is a real number; '' is a real but empty text value. They behave differently in comparisons and functions.

Q12. What does DISTINCT do?

A. Removes duplicate values from the result so you get each unique value once.

Q13. Difference between COUNT(*) and COUNT(column)?

A. COUNT(*) counts all rows; COUNT(column) counts only rows where that column is not NULL.

Q14. What is GROUP BY used for?
A. It buckets rows by a column so aggregate functions run per group, e.g. count of customers per city.

Q15. Difference between UNION and UNION ALL?
A. UNION stacks two result sets and removes duplicates; UNION ALL keeps duplicates and is faster.

Q16. What is a subquery?
A. A query nested inside another; the inner query runs first and its result feeds the outer query.

Q17. What is the difference between IN and EXISTS?
A. IN checks a value against a list/subquery result; EXISTS checks whether a related subquery returns any row at all, often faster on large correlated checks.

Q18. What are the ACID properties?
A. Atomicity, Consistency, Isolation, Durability — the guarantees that make transactions reliable.

Q19. What is a transaction?
A. A group of statements treated as one unit that either all commit or all roll back.

Q20. What is an index and why does it matter to QA?
A. A structure that speeds up lookups. In performance testing it explains why a query is fast or slow; missing indexes are a common cause of slow pages.

Q21. What is a view?
A. A saved SELECT that behaves like a virtual table, handy for reusing a complex query.

Q22. What is normalization (in one line)?
A. Organising data to reduce redundancy by splitting it into related tables — the reason JOINs exist.

Q23. What is denormalization?
A. Deliberately adding redundancy back (e.g. duplicated columns) to make reads faster, trading storage and update-complexity for speed.

Q24. What's the difference between CHAR and VARCHAR?
A. CHAR is fixed length (padded); VARCHAR is variable length (stores only what's needed). Mismatched types are a common data-mapping bug.

Q25. How do you find duplicate rows?
A. GROUP BY the column(s) that should be unique and add HAVING COUNT(*) > 1.

Q26. How would you verify a record was inserted from the UI?
A. Run a SELECT filtering on a unique field you entered (like the email) and confirm exactly one matching row with the right values.

Q27. What is referential integrity?
A. The rule that a foreign key must point to an existing primary key — no orphan child rows. Broken referential integrity is a data bug.

Q28. What is a candidate key vs composite key?
A. A candidate key is any column set that could be the primary key; a composite key is a primary key made of two or more columns together.

Q29. What order does SQL logically execute in?

A. FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT, even though you type SELECT first.

Q30. How do you get the second-highest value?

A. One way: SELECT MAX(total) FROM orders WHERE total < (SELECT MAX(total) FROM orders). Or use ORDER BY DESC with LIMIT/OFFSET.

Q31. What is a stored procedure?

A. A saved, reusable block of SQL you can call by name, sometimes itself the object under test.

Q32. What's the risk of an UPDATE without WHERE?

A. It updates every row in the table. Always preview the same filter with a SELECT first, and use a transaction on real data.

Q33. How do you test data migration / ETL?

A. Compare source vs target: row counts (COUNT(*)), spot-check values, check no truncation or type changes, and confirm no rows dropped or duplicated.

Q34. What is the difference between a clustered and non-clustered index (bonus)?

A. A clustered index sets the physical order of the rows (one per table); a non-clustered index is a separate lookup pointing to the rows (many allowed).

Q35. Why does a tester need SQL at all?

A. To verify at the database level what the app claims at the UI level — setting up test data, and confirming data integrity after every create, update, or delete.

Cheat-sheet: the patterns you'll reuse

```
Read           SELECT cols FROM t WHERE cond ORDER BY col LIMIT n;
Count per group SELECT g, COUNT(*) FROM t GROUP BY g HAVING COUNT(*)>k;
Join            SELECT ... FROM a JOIN b ON b.fk = a.pk;
Find gaps       SELECT ... FROM a LEFT JOIN b ON ... WHERE b.pk IS NULL;
Duplicates      SELECT c, COUNT(*) FROM t GROUP BY c HAVING COUNT(*)>1;
Verify insert   SELECT * FROM t WHERE unique_col = 'value';
Safe change     BEGIN; UPDATE/DELETE ... WHERE ...; COMMIT | ROLLBACK;
```

You're viva-ready

If you can explain PK vs FK, WHERE vs HAVING, INNER vs LEFT JOIN, the NULL rule, and how you'd verify a UI action in the database — you've covered what most QA interviews actually ask.